

ADR and Polly Requirement Explanation

Requirement: "Write 3 ADRs for decisions made so far; add Polly Circuit Breaker + Retry on an outbound call."

Result: Both parts of the requirement are fulfilled. The repository contains three accepted Architecture Decision Records, and the Postbound email HTTP call is protected by Polly retry, circuit-breaker, and timeout policies.

1. The three ADRs

ADR 001: Use URL-segment API versioning

`docs/adr/001-url-api-versioning.md`

Explains why public endpoints use routes such as `/api/v1/...` and `/api/v2/...`. It covers concurrent version support, v1 deprecation and sunset headers, separate Swagger documents, alternatives considered, and the consequences of placing the version in the URL.

ADR 002: Return Problem Details for API failures

`docs/adr/002-problem-details-errors.md`

Documents the decision to standardize failures with `application/problem+json`. It describes global exception handling, validation errors, safe exception mapping, trace identifiers, and consistent client-facing error contracts.

ADR 003: Process bulk notifications asynchronously and protect outbound calls

`docs/adr/003-background-bulk-notifications-and-resilience.md`

Documents the decision to queue bulk notifications for background processing and to protect the external email provider with timeout, retry, and circuit-breaker policies. It also records limitations such as in-memory durability and the duplicate-delivery risk of retrying HTTP POST requests.

Each ADR contains the normal decision-record elements: context, decision drivers, alternatives, selected decision, tradeoffs, validation criteria, and conditions under which the decision should be revisited.

2. The protected outbound call

The external request is made by `Infrastructure/Email/PostboundEmailSender.cs`. The class is a typed HTTP client implementing `IEmailSender`.

```
using var request = new HttpRequestMessage(HttpMethod.Post, "emails")
{
    Content = JsonContent.Create(new { to = recipientEmail, subject, body })
};
```

```
request.Headers.Authorization =  
    new AuthenticationHeaderValue("Bearer", settings.ApiKey);  
  
using var response = await httpClient.SendAsync(request, cancellationToken);  
response.EnsureSuccessStatusCode();
```

Because this `HttpClient` is created by the typed-client registration in `Program.cs`, its `SendAsync` call automatically passes through the registered Polly handlers.

3. Retry policy

The retry policy is defined in `Infrastructure/Email/EmailResiliencePolicies.cs`.

```
HttpPolicyExtensions  
    .HandleTransientHttpError()  
    .Or<TimeoutRejectedException>()  
    .WaitAndRetryAsync(  
        retryCount: options.RetryCount,  
        sleepDurationProvider: retryAttempt =>  
            TimeSpan.FromMilliseconds(  
                options.InitialRetryDelayMilliseconds  
                * Math.Pow(2, retryAttempt)),  
        onRetry: ...);
```

`HandleTransientHttpError()` handles network failures, HTTP 408, and HTTP 5xx responses. `TimeoutRejectedException` adds failures produced by the Polly timeout policy.

With the current configuration, Polly performs three retries with exponential backoff:

Retry	Calculation	Delay
1	200×2^1	400 ms
2	200×2^2	800 ms
3	200×2^3	1,600 ms

One logical send can therefore make four physical provider attempts: the original attempt plus three retries.

4. Circuit-breaker policy

The circuit breaker is also defined in `Infrastructure/Email/EmailResiliencePolicies.cs`.

```
HttpPolicyExtensions  
    .HandleTransientHttpError()  
    .Or<TimeoutRejectedException>()  
    .CircuitBreakerAsync(  
        handledEventsAllowedBeforeBreaking:
```

```

        options.CircuitBreakerFailures,
        durationOfBreak:
            TimeSpan.FromSeconds(options.CircuitBreakDurationSeconds),
        onBreak: ...,
        onReset: ...,
        onHalfOpen: ...);

```

With the configured threshold of three failures and a 30-second break:

1. The closed circuit initially permits calls.
2. After three failed logical sends, the circuit opens.
3. Calls received while it is open fail immediately without contacting the provider.
4. After 30 seconds, the circuit becomes half-open and allows a trial call.
5. A successful trial resets the circuit; a failed trial opens it again.

5. Why the policy provider is a singleton

`EmailResiliencePolicyProvider` constructs and owns the policy instances. It is registered as a singleton in `Program.cs`:

```
builder.Services.AddSingleton<EmailResiliencePolicyProvider>();
```

A circuit breaker must retain its failure count and open/closed state between application requests. If a new breaker were created for every email, its count would continually reset and it might never open.

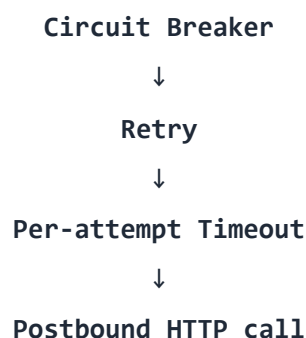
6. Policy registration and execution order

The typed client and policies are registered in `Program.cs`:

```

builder.Services
    .AddHttpClient<PostboundEmailSender>(...)
    .AddPolicyHandler((services, _) =>
        services.GetRequiredService<EmailResiliencePolicyProvider>().CircuitBreaker)
    .AddPolicyHandler((services, _) =>
        services.GetRequiredService<EmailResiliencePolicyProvider>().Retry)
    .AddPolicyHandler((services, _) =>
        services.GetRequiredService<EmailResiliencePolicyProvider>().Timeout);

```



Every attempt receives its own timeout. The retry policy can repeat a timed-out or transiently failed attempt. The outer circuit breaker sees the final outcome of the complete retry sequence, so it counts failed logical sends rather than individual HTTP attempts.

7. Application call path

The application depends on `IEmailSender`, keeping provider-specific HTTP logic outside the service layer. `Service/Notification/NotificationDispatcher.cs` saves the notification intent and calls the abstraction:

```
await notificationRepository.SaveChangesAsync(cancellationToken);

var deliveryOutcome = await emailSender.SendAsync(
    recipientEmail,
    subject,
    body,
    cancellationToken);
```

NotificationDispatcher → **IEmailSender** → **PostboundEmailSender**
→ **Circuit Breaker** → **Retry** → **Timeout** → **HttpClient.SendAsync**

8. Configuration and current runtime behavior

The values are configured under `Postbound` in `appsettings.json`:

```
"Postbound": {
  "BaseUrl": "https://api.postbound.com/",
  "ApiKey": "",
  "Enabled": false,
  "RequestTimeoutSeconds": 10,
  "RetryCount": 3,
  "InitialRetryDelayMilliseconds": 200,
  "CircuitBreakerFailures": 3,
  "CircuitBreakDurationSeconds": 30
}
```

The matching validated settings class is `Infrastructure/Email/PostboundOptions.cs`.

Important: The resilience implementation is fully wired, but the provider is disabled by default. With `Enabled: false`, the sender returns `NotAttempted` and makes no network request. A verified provider contract, API key, and `Enabled: true` are required for a real outbound call.